

CLOUD WATCH & CLOUD TRAIL TASK

Importance of CloudTrail & CloudWatch:

- AWS CloudTrail is essential for security and compliance, as it records every API call made in an AWS account, enabling full visibility into user activity, resource changes, and potential unauthorized actions. It helps detect security breaches, supports auditing, and provides evidence for forensic analysis.
- AWS CloudWatch is crucial for real-time monitoring of AWS resources and applications. It tracks performance metrics, logs, and system health to ensure smooth operations. CloudWatch also enables automated alerts (via alarms) and proactive actions to resolve issues like high CPU usage or service failures.
- Together, CloudTrail and CloudWatch form the backbone of AWS monitoring and security, ensuring transparency, operational efficiency, and rapid incident response.

Tool used for this Task:

- S3 bucket (Simple storage service)
- CloudTrail
- CloudWatch

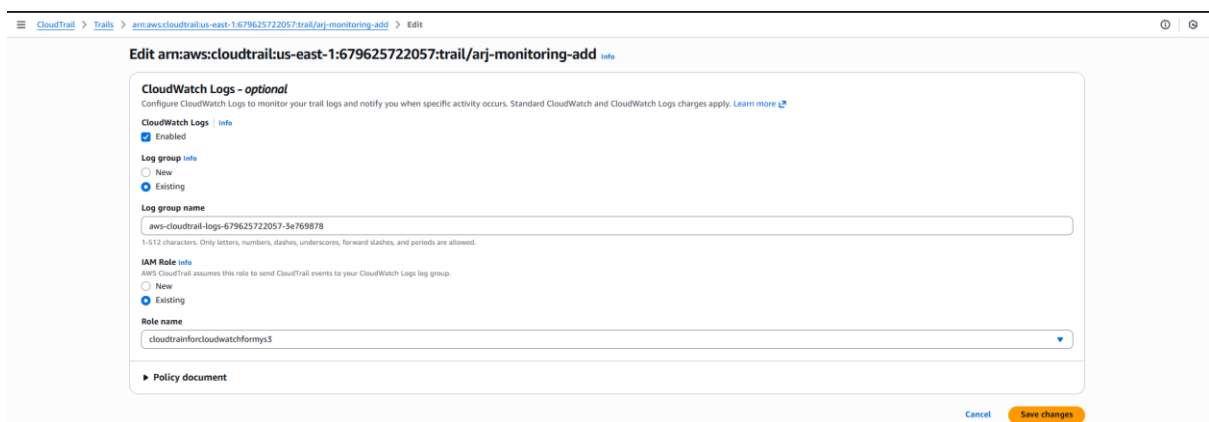
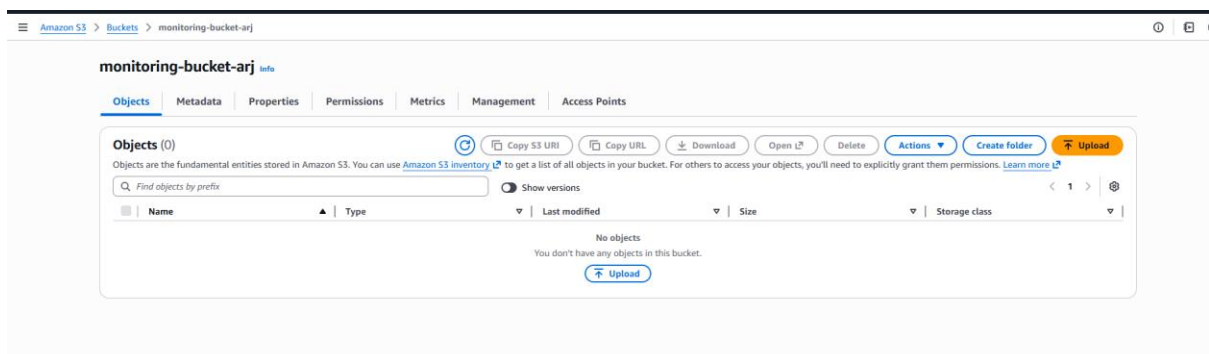
Takeaway:

- CloudTrail enables complete visibility and auditing of all AWS account activity, ensuring security and compliance. CloudWatch provides real-time monitoring, metrics, and automated alerts to maintain performance and detect issues proactively.

1) Enable cloudtrail monitoring and store the events in s3 and cloudwatch log events.

Steps:

- Opened AWS CloudTrail console and selected *Create Trail* to begin configuration.
 - Named the trail and enabled it as a multi-region trail to capture events across all regions.
 - Created a new S3 bucket (e.g., *monitoring-bucket-arj*) to store CloudTrail logs securely.
 - Enabled Log file SSE-KMS encryption (optional) to protect logs using AWS KMS.
 - Enabled CloudWatch Logs integration, where a new Log Group and IAM role were automatically created.
 - Reviewed and confirmed the configuration, then created the trail, after which CloudTrail began logging to both S3 and CloudWatch Logs.
- Created s3 bucket as you can see in the image I have disabled public block and created public s3 bucket



- Created cloud trail with existing s3 bucket and provided my s3 bucket name.

CloudTrail > Trails > [arn:aws:cloudtrail:us-east-1:679625722057:trail/arj-monitoring-add](#) > Edit

Edit arn:aws:cloudtrail:us-east-1:679625722057:trail/arj-monitoring-add

General details
A trail created in the console is a multi-region trail. [Learn more](#)

Trail name
Enter a display name for your trail.
arj-monitoring-add
3-128 characters. Only letters, numbers, periods, underscores, and dashes are allowed.

☐ Enable for all accounts in my organization
To review accounts in your organization, open AWS Organizations. [See all accounts](#)

Storage location [Info](#)
☐ Create new S3 bucket
Create a bucket to store logs for this trail.
☒ Use existing S3 bucket
Choose an existing bucket to store logs for this trail.

Trail log bucket name
Enter a new S3 bucket name and folder (prefix) to store your logs. Bucket names must be globally unique.
monitoring-bucket-arj [Browse](#)

Prefix - optional
prefix
Logs will be stored in monitoring-bucket-arj/Prefix/679625722057

Log file SSE-KMS encryption [Info](#)
☐ Enabled

Additional settings
Log file validation [Info](#)
☒ Enabled
SNS notification delivery [Info](#)
☐ Enabled

- Enable cloud watch while creating cloud trail event.

CloudTrail > Trails > [arn:aws:cloudtrail:us-east-1:679625722057:trail/arj-monitoring-add](#)

arj-monitoring-add

[Delete](#) [Stop logging](#)

General details [Edit](#)

Trail logging [Logging](#)
 Trail name: arj-monitoring-add
 Multi-region trail: Yes
 Apply trail to my organization: Not enabled

Trail log location [monitoring-bucket-arj/Prefix/679625722057](#)
 Last log file delivered: -
 Log file SSE-KMS encryption: Not enabled

Log file validation
 Enabled
 Last file validation delivered: December 07, 2025, 09:40:02 (UTC+05:30)

SNS notification delivery
 Disabled
 Last SNS notification: -

CloudWatch Logs [Edit](#)
 Log group: aws-cloudtrail-logs-679625722057-3e769878
 IAM Role: arn:aws:iam::679625722057:role/service-role/cloudtrailforcloudwatchformys3

Tags [Manage tags](#)

- Logs are created as we can see in this image

Amazon S3 > Buckets > [General purpose buckets](#) > [Table buckets](#) > [Vector buckets](#) > [New](#)

07/

[Copy S3 URI](#)

Objects [Properties](#)

Objects (5)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

[Find objects by prefix](#) [Show versions](#)

☐	Name	Type	Last modified	Size	Storage class
☐	679625722057_CloudTrail_us-east-2_20251207T0415Z_Ninfg6Ovj4Z9g27m8.json.gz	gz	December 7, 2025, 09:42:24 (UTC+05:30)	3.9 KB	Standard
☐	679625722057_CloudTrail_us-east-2_20251207T0420Z_hws8tWzp5sr13Dzbj5on.gz	gz	December 7, 2025, 09:47:24 (UTC+05:30)	5.3 KB	Standard
☐	679625722057_CloudTrail_us-east-2_20251207T0420Z_xNuA2xwF9oUElwIX.json.gz	gz	December 7, 2025, 09:52:13 (UTC+05:30)	7.5 KB	Standard
☐	679625722057_CloudTrail_us-east-2_20251207T0440Z_bM8NkZy0B876ck7js.on.gz	gz	December 7, 2025, 10:12:24 (UTC+05:30)	2.7 KB	Standard
☐	679625722057_CloudTrail_us-east-2_20251207T0450Z_rNPbdUvMK3uRNdW.json.gz	gz	December 7, 2025, 10:18:04 (UTC+05:30)	2.1 KB	Standard

2) Enable SNS for cloudtrail to send alert on email.

Steps:

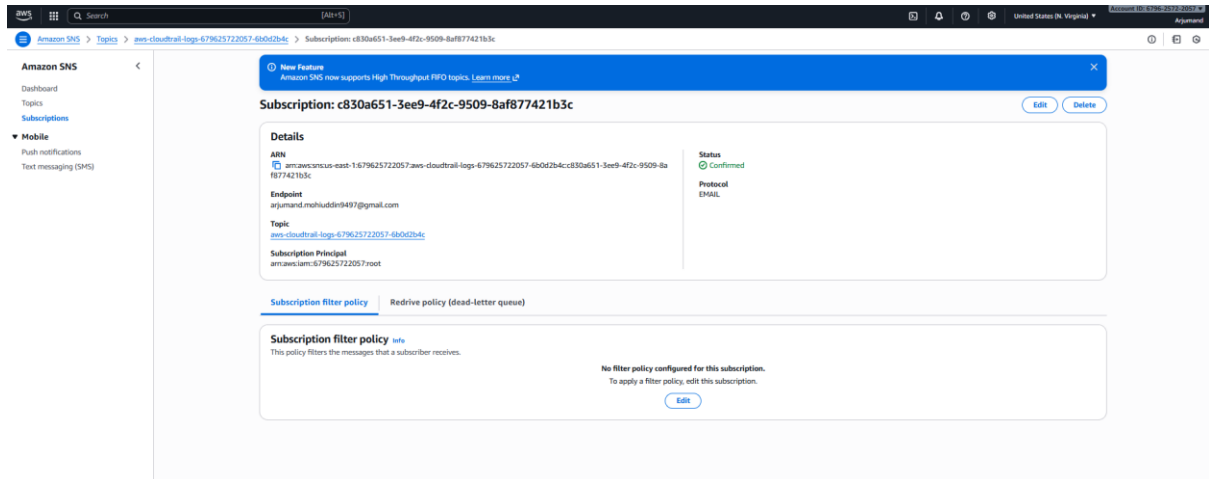
- Created SNS Topic
 - Opened SNS → Create Topic
 - Type: Standard
 - Name: cloudtrail-alerts
 - Step 2: Subscribe Email
 - Inside the topic, click Create Subscription
 - Protocol → Email
 - Enter your email → Click Create
 - Check your inbox and Confirm subscription
- Created new topic in sns path: SNS in aws console > create new topic
 - Selected all default settings and created one sns

The screenshot shows the 'Edit topic' page in the AWS SNS console. At the top, there is a blue banner with a 'New Feature' notification. Below the banner, the breadcrumb navigation shows 'Amazon SNS > Topics > aws-cloudtrail-logs-679625722057-6b0d2b4c > Edit topic'. The main heading is 'Edit aws-cloudtrail-logs-679625722057-6b0d2b4c'. The 'Details' section shows the topic name 'aws-cloudtrail-logs-679625722057-6b0d2b4c' and type 'Standard'. There is a 'Display name - optional' field with the value 'My_new_topic_sns'. Below this are several expandable sections for optional settings: 'Encryption - optional', 'Access policy - optional', 'Data protection policy - optional', 'Delivery policy (HTTP/S) - optional', and 'Message delivery status logging - optional'. Each section has a brief description of its function.

- Created subscription and selected mail and added mail path sns <subscription

The screenshot shows the 'Create subscription' page in the AWS SNS console. At the top, there is a blue banner with a 'New Feature' notification. Below the banner, the breadcrumb navigation shows 'Amazon SNS > Subscriptions > Create subscription'. The main heading is 'Create subscription'. The 'Details' section contains a 'Topic ARN' field with the value 'arn:aws:sns:us-east-1:679625722057:aws-cloudtrail-logs-679625722057-6b0d2b4c'. The 'Protocol' is set to 'Email'. The 'Endpoint' field contains the email address 'arjmand.mohiuddin@997-tgokails'. Below this is a confirmation message: 'After your subscription is created, you must confirm it.' At the bottom, there are two expandable sections for optional settings: 'Subscription filter policy - optional' and 'Redrive policy (dead-letter queue) - optional'. At the bottom right, there are 'Cancel' and 'Create subscription' buttons.

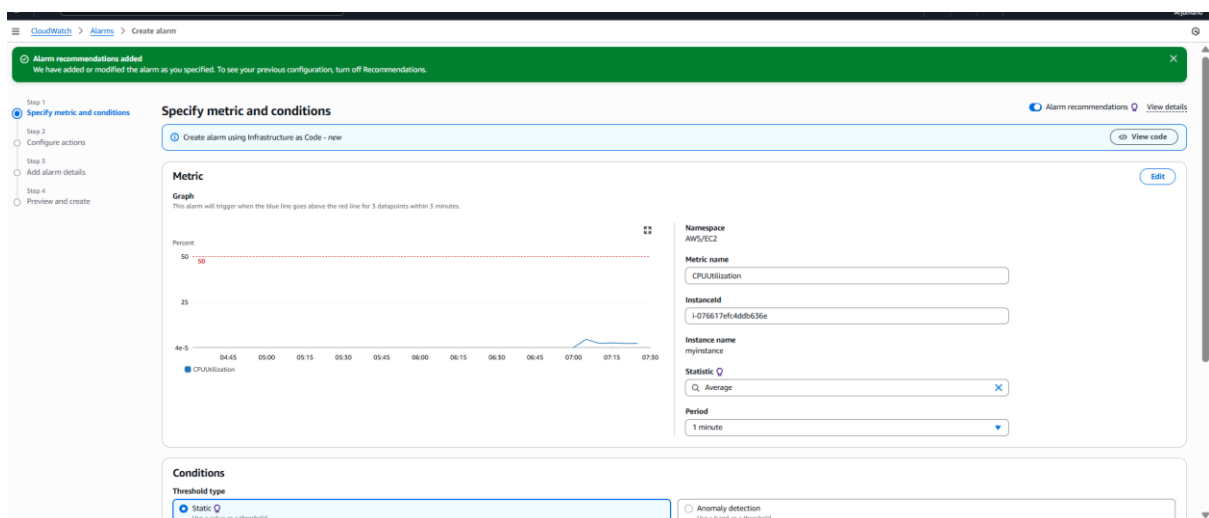
- Confirmed mail subscription and status went to confirmed



3) Configure cloud watch monitoring and record the cpu utilization and other metrics of ec2. b

Steps:

- Opened **CloudWatch** → **Metrics** and selected your EC2 instance's **CPUUtilization** metric.
 - Then navigated to **CloudWatch** → **Alarms** → **Create alarm** (because the Metrics view does not show the alarm option).
 - Selected the EC2 metric again from **EC2** → **Per-Instance Metrics**.
 - Configured the alarm condition (e.g., CPU > 80%) and added your **SNS topic** for notifications.
 - Named the alarm and clicked **Create alarm** to enable CPU monitoring and alerts
- I opened the cloudwatch navigate to alarm, created alarm then selected metric



- Selected static option in under notification and given parameters 50 greater and equal to and created

- Created alarm successfully

Name	State	Last state update (UTC)	Conditions	Actions
AWS/EC2 CPUUtilization InstanceId=i-076617ef-4dd6b35e	Insufficient data	2025-12-07 07:34:00	CPUUtilization >= 50 for 3 datapoints within 5 minutes	Actions enabled

4) Create one alarm to send alert to email if the cpu utilization is more than 70 percent.

Steps:

- Logged into your EC2 instance using SSH and created a Bash script (`load.sh`) to generate CPU load.
 - Added the infinite loop code inside the script to push CPU usage to ~70–90%.
 - Made the script executable using `chmod +x load.sh`.
 - Ran the script to generate high CPU utilization and verify your CloudWatch alarm.
 - Stopped the load by killing the running processes using `kill -f load.sh` and confirmed CPU dropped back to normal.
- First I have edited 50% usage to 70% usage in previous task I have created my ec2 instance cpu utilization alert
 - Then I have provided some load to my ece using bash script

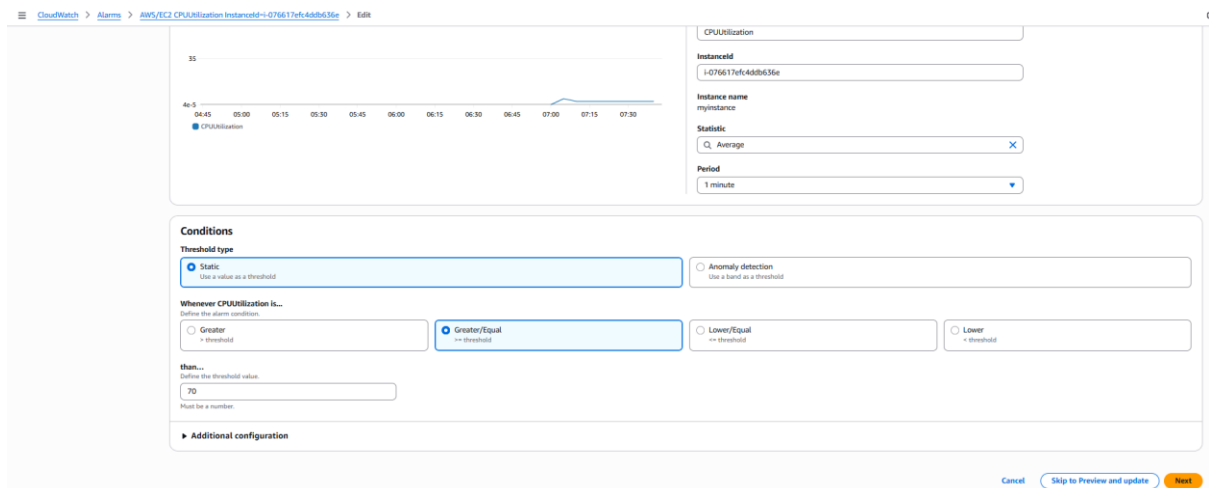
```
#!/bin/bash
echo "Starting CPU load... Press CTRL+C to stop."

# Number of CPU cores
CORES=$(nproc)

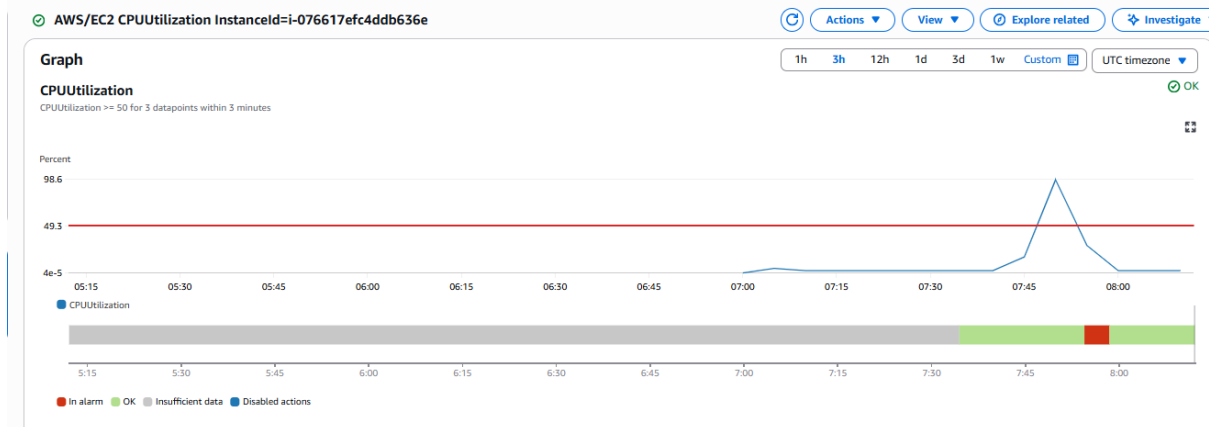
# Run a heavy loop on each core
for i in $(seq 1 $CORES)
do
    while :; do :; done &
done

echo "CPU load started on $CORES cores."
|
```

- In the image we can see the utilization about to start



- Utilization reached 50% and above then I got mail



- I got the mail regarding the utilization

< All Mail



My_new_topic_sns

1:24 PM

To: arjumand.mohiuddin9497@gmail.com



ALARM: "AWS/EC2 CPUUtilization Instanceld=i-076617efc4d- db636e" in US East (N. Virginia)

You are receiving this email because your
Amazon CloudWatch Alarm "AWS/EC2
CPUUtilization

Instanceld=i-076617efc4ddb636e" in the US
East (N. Virginia) region has entered the ALARM
state, because "Threshold Crossed: 1 out of the
last 3 datapoints [98.56473813927154
(07/12/25 07:50:00)] was greater than or equal
to the threshold (50.0) (minimum 3 datapoints
for OK -> ALARM transition)." at "Sunday 07
December, 2025 07:54:30 UTC".

View this alarm in the AWS Management
Console:

[https://us-east-1.console.aws.amazon.com/
cloudwatch/deeplink.js?region=us-
east-1#alarmsV2:alarm/](https://us-east-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-east-1#alarmsV2:alarm/AWS%2FEC2%2FCPUUtilization%2FInstanceld=i-076617efc4ddb636e)

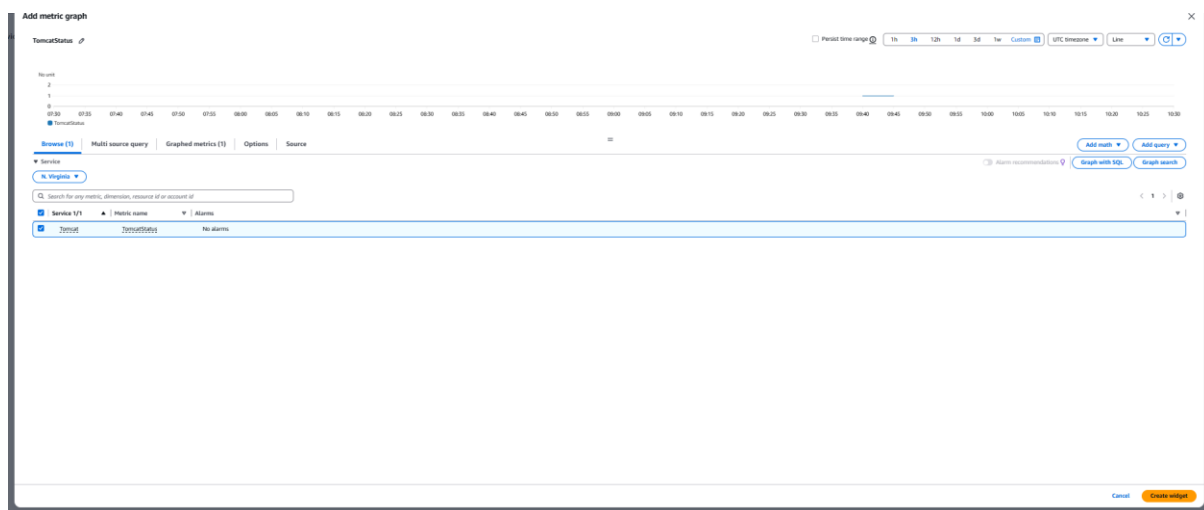
[AWS%2FEC2%2FCPUUtilization%2FInstanceld=i-076617efc4ddb636e](https://us-east-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-east-1#alarmsV2:alarm/AWS%2FEC2%2FCPUUtilization%2FInstanceld=i-076617efc4ddb636e)

5) Create Dashboard and monitor tomcat service whether it is running or not and send the alert.

Steps

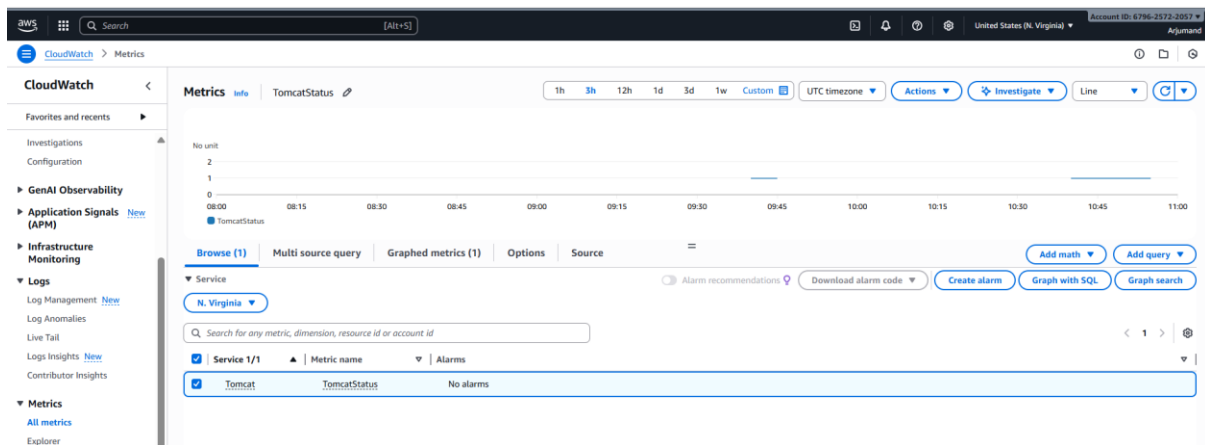
- 1 Create `tomcat.bash` script to check Tomcat process and push `TomcatStatus` to CloudWatch.
 - 2 Make the script executable: `chmod +x tomcat.bash`.
 - 3 Test AWS CLI metric manually to confirm it appears in CloudWatch.
 - 4 Add cron job: `* * * * * /home/ec2-user/tomcat.bash >> /home/ec2-user/tomcat.log 2>&1`.
 - 5 Create CloudWatch alarm (`TomcatStatus ≤ 0`) and add it to Dashboard.
- Wrote a script for tomcat to check Tomcat monitoring process

```
#!/bin/bash
# Check if Tomcat process is running
if pgrep -f "org.apache.catalina.startup.Bootstrap" > /dev/null
then
    STATUS=1 # Tomcat is running
else
    STATUS=0 # Tomcat is stopped
fi
# Push custom metric to CloudWatch
aws cloudwatch put-metric-data \
--metric-name TomcatStatus \
--namespace "TomcatMonitoring" \
--value $STATUS \
--dimensions "Service=Tomcat"
```



- Added tomcat monitoring in crontab using crontab -e

```
ec2-user@ip-172-31-22-154:~
$ /home/ec2-user/tomcat.bash >> /home/ec2-user/tomcat.log 2>&1
```



6) Create Dashboard and monitor nginx service to send the alert if nginx is not running.

Steps:

- 1 Create `nginx-monitor.sh` script to check NGINX service and send `NginxStatus` to CloudWatch.
- 2 Make it executable: `chmod +x nginx-monitor.sh`.
- 3 Test sending metric manually using AWS CLI to verify visibility in CloudWatch.
- 4 Add cron job: `* * * * * /home/ec2-user/nginx-monitor.sh >> /home/ec2-user/nginx-monitor.log 2>&1`.
- 5 Create CloudWatch alarm (`NginxStatus ≤ 0`) and add widget in Dashboard.

- Wrote a script for nginx to check nginx monitoring process

```

ec2-user@ip-172-31-22-154:~$ cat nginx-monitor.sh
#!/bin/bash

# Check if NGINX is running
if systemctl is-active --quiet nginx; then
    STATUS=1 # Running
else
    STATUS=0 # Not running
fi

# Send metric to CloudWatch
aws cloudwatch put-metric-data \
    --metric-name NginxStatus \
    --namespace "NginxMonitoring" \
    --value $STATUS \
    --dimensions "Service=NGINX"

```

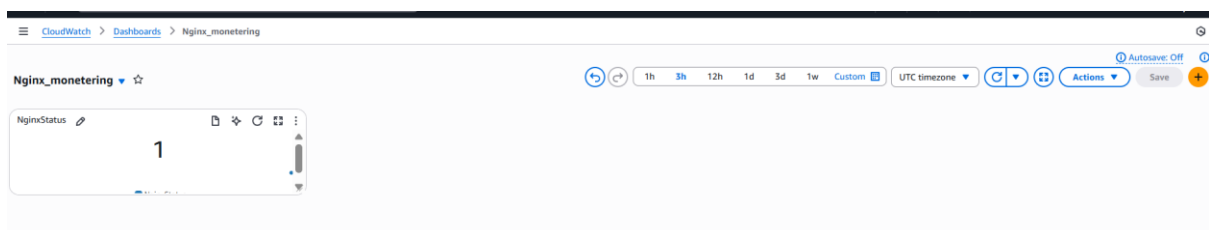
- Nginx script is added in Crontab for monitoring nginx working status

```

ec2-user@ip-172-31-22-154:~$ crontab -l
* * * * * /home/ec2-user/tomcat.bash >> /home/ec2-user/tomcat.log 2>&1
* * * * * /home/ec2-user/nginx-monitor.sh >> /home/ec2-user/nginx-monitor.log 2>&1

```

- Created dashboard for nginx monitoring



- Added alert in aws console < cloudwatch. The created one alert to monitor nginx working status